

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:

Training : (N_train, 128, 9)

Validation : (N_val, 128, 9)

Testing : (N_test, 128, 9)

Number of classes: 6
Number of features per time step: 9
Sequence length: 128

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

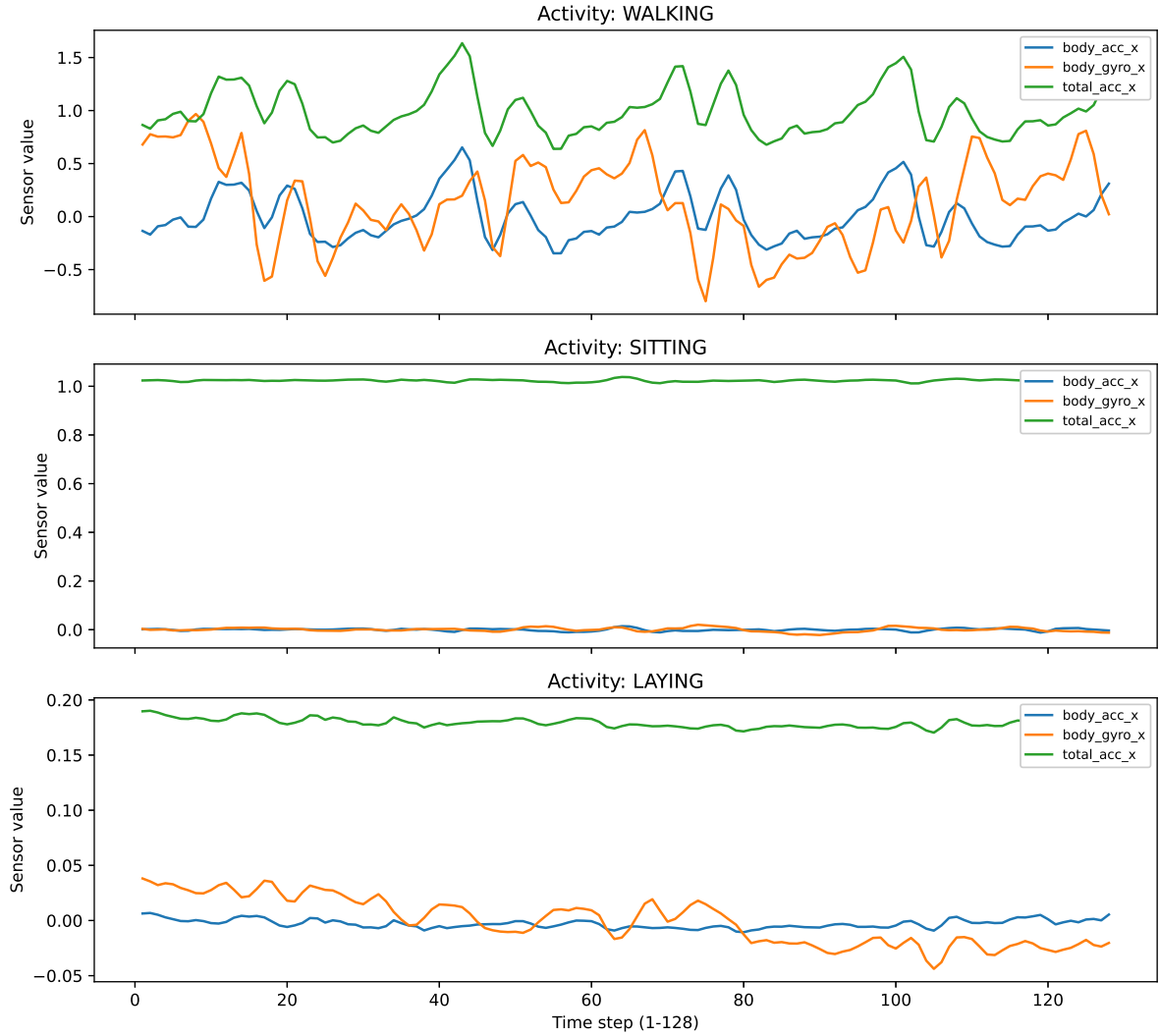


Figure 1: Sensor Signal versus Time

Required inference:

1. What temporal pattern is visible?
2. Which activities exhibit similar patterns?
3. Why is the ordering of measurements important?

The sensor signals show distinct temporal variations for different activities. Activities such as walking exhibit more periodic fluctuations, while stationary activities show relatively stable patterns. This occurs because different physical movements produce different sensor patterns over time. Hence, the temporal ordering of sensor measurements is important for activity classification.

7. Vanilla RNN Architecture

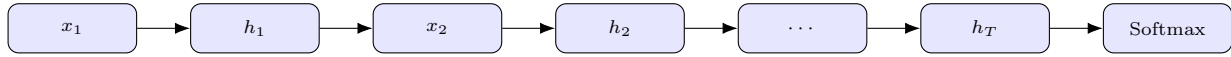
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

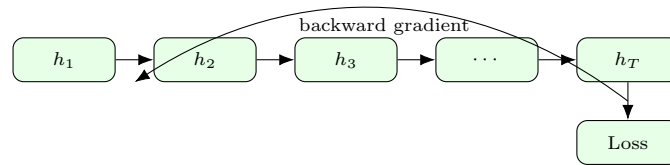
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

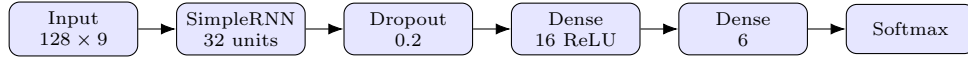
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

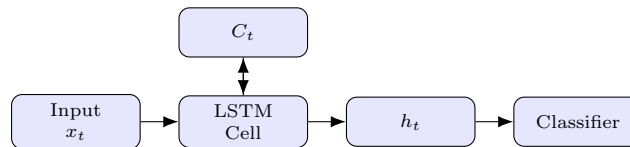
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

The cell state acts as a memory pathway that carries important information across time steps. The forget gate removes irrelevant information, the input gate adds useful new information, and the output gate controls what information is passed to the next layer. Thus, LSTM can retain important long-term dependencies while discarding unnecessary information over long sequences.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

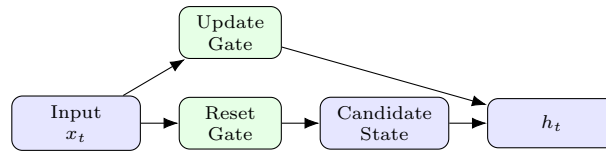
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

Table 1: Comparison of RNN, LSTM and GRU

Model	Number of Gates	State Representation
RNN	No gates	Hidden state
LSTM	3 gates	Cell state + Hidden state
GRU	2 gates	Hidden state

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

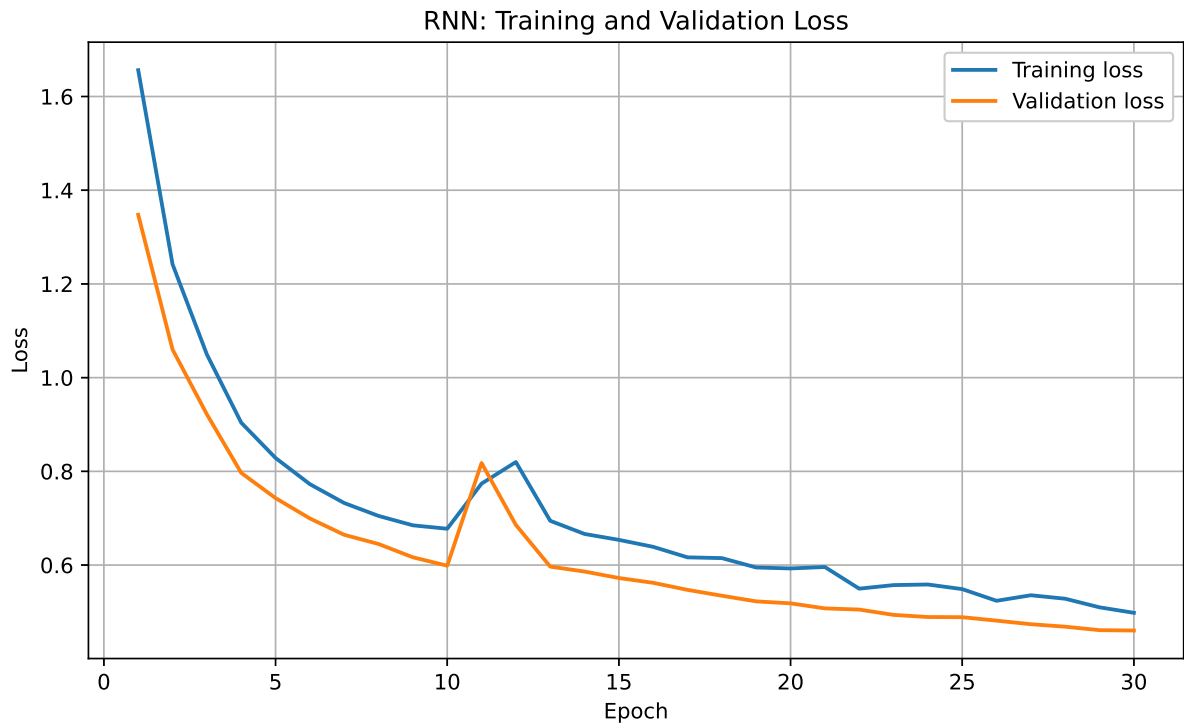


Figure 2: RNN Training and Validation Loss

The training and validation loss decrease with epochs and gradually stabilize, indicating convergence. The validation loss remains higher than the training loss, suggesting a generalization gap. The RNN learns the sequence patterns but has limited ability to capture long-term dependencies.

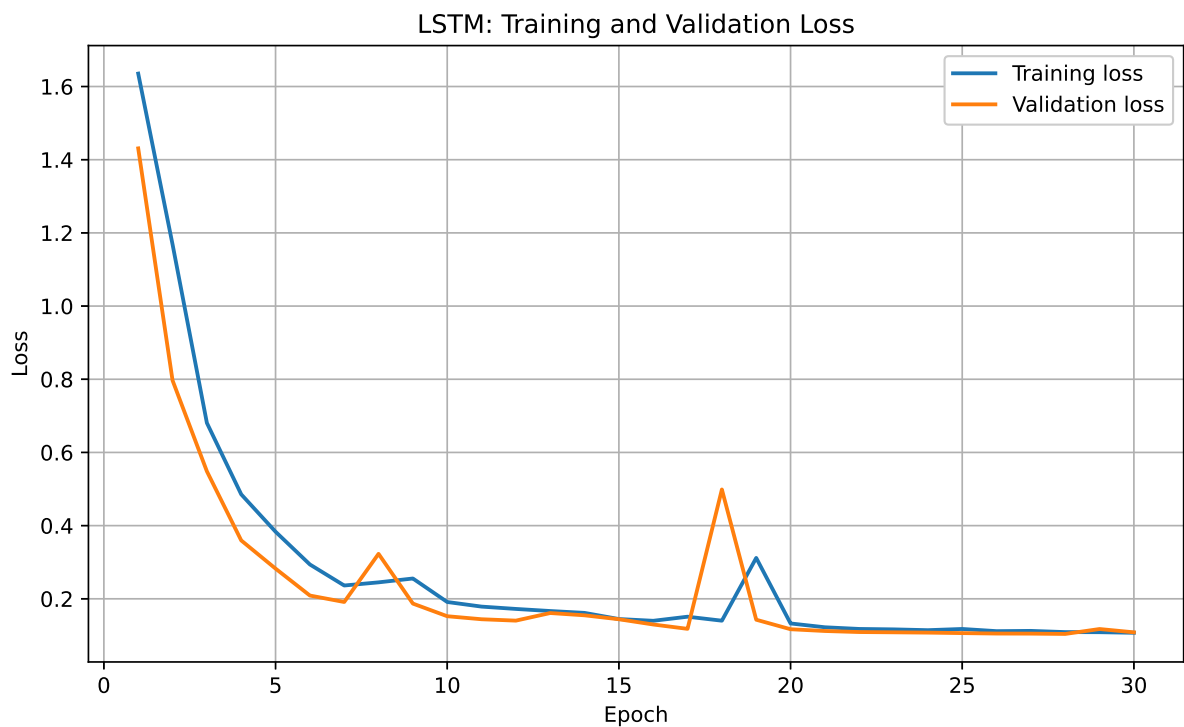


Figure 3: LSTM Training and Validation Loss

Both training and validation loss decrease and stabilize, showing good convergence. The relatively close curves indicate effective generalization. The LSTM gates and cell state help retain relevant information over longer sequences.

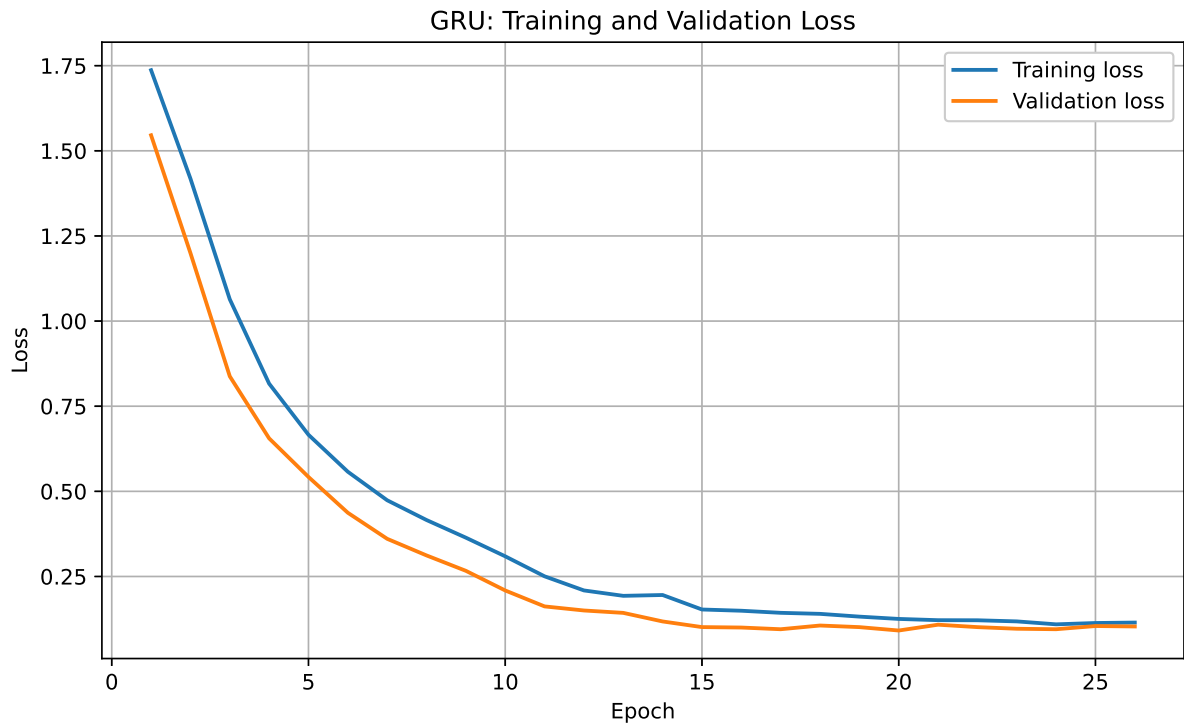


Figure 4: GRU Training and Validation Loss

Training and validation loss decrease and eventually stabilize, indicating convergence. The curves show that GRU effectively learns temporal dependencies while using a simpler gated structure than LSTM.

Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

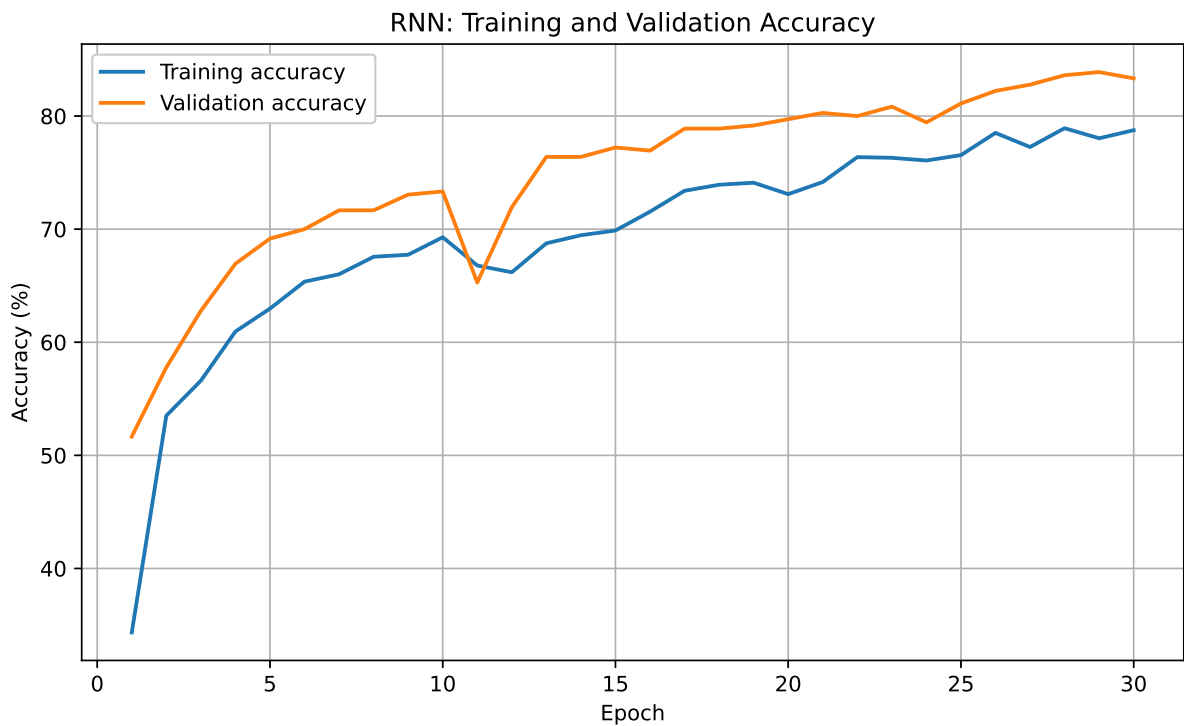


Figure 5: RNN Training and Validation Accuracy

Training and validation accuracy increase with epochs and then level off. The lower final accuracy compared with the gated models indicates that the basic RNN has greater difficulty learning complex temporal dependencies.

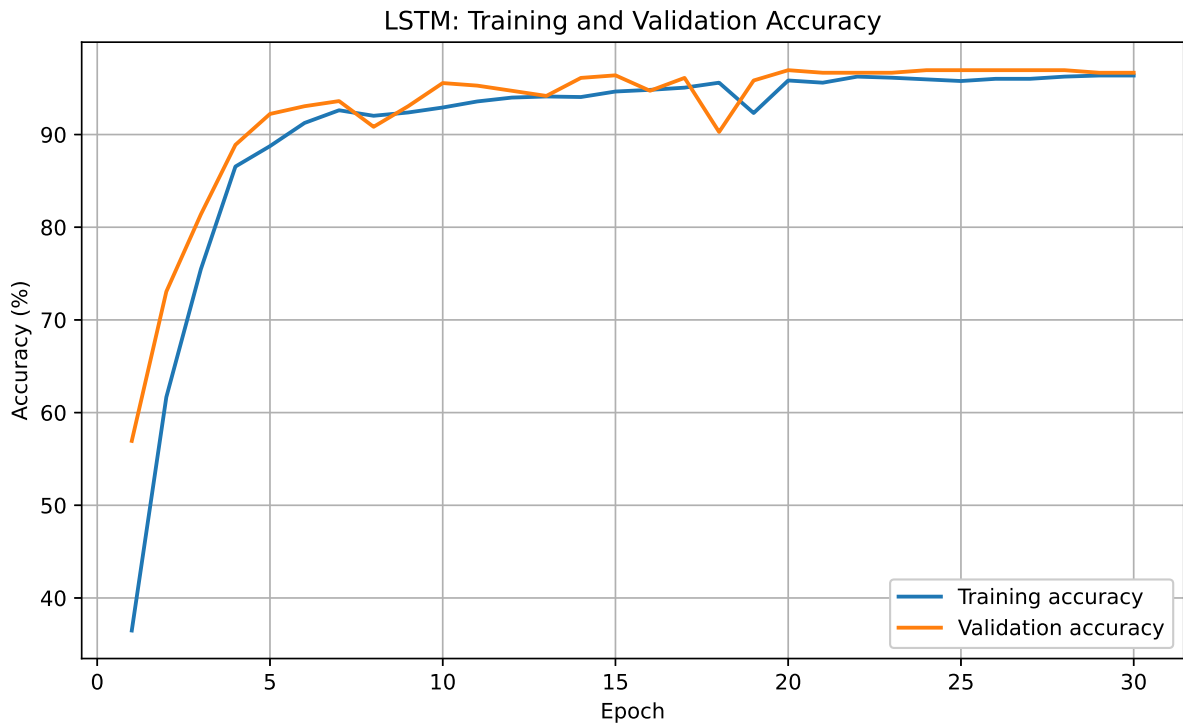


Figure 6: LSTM Training and Validation Accuracy

Training and validation accuracy increase rapidly and stabilize at a high level. The small gap between the curves indicates good generalization and effective sequence learning.

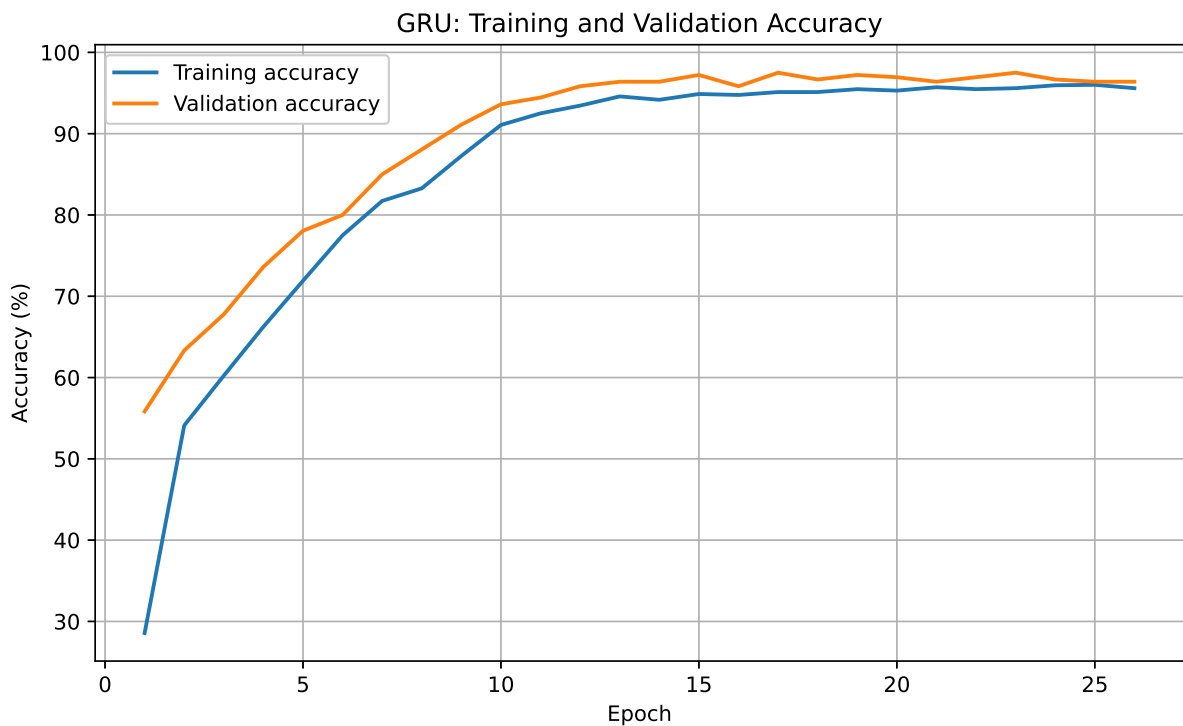


Figure 7: GRU Training and Validation Accuracy

Accuracy increases and converges to a high level, with training and validation performance remaining close. This indicates that GRU can learn the temporal patterns effectively with its simpler gating mechanism.

Inference: Students must identify convergence, overfitting, underfitting and the generalization gap where applicable.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	76.67	96.39	96.39
Macro Precision (%)	75.04	96.67	96.67
Macro Recall (%)	74.71	96.66	96.66
Macro F1 (%)	74.63	96.66	96.66
Parameters	1974	6006	4758
Training Time (s)	43.95	79.16	115.32

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

Generate a separate confusion matrix for each model.

Students must identify:

- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

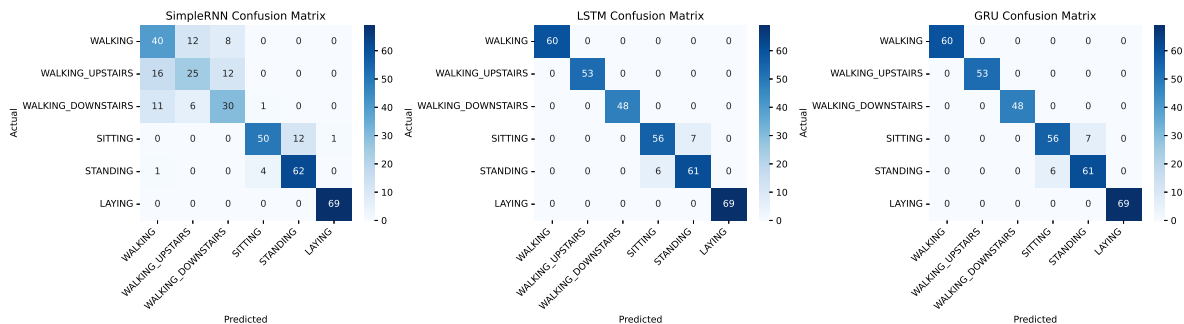


Figure 8: Confusion Matrix

The confusion matrix shows correct predictions along the diagonal and misclassifications in the off-diagonal cells. The errors mainly occur between activities with similar motion patterns, while activities with more distinctive sensor patterns are recognized more accurately. Therefore, the matrix provides class-wise error information that overall accuracy alone cannot reveal. The exact confused activity pairs should be reported from the student's observed matrix.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters			
Training time			
Test F1-score			

Plot 5: Model Performance Comparison

Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.

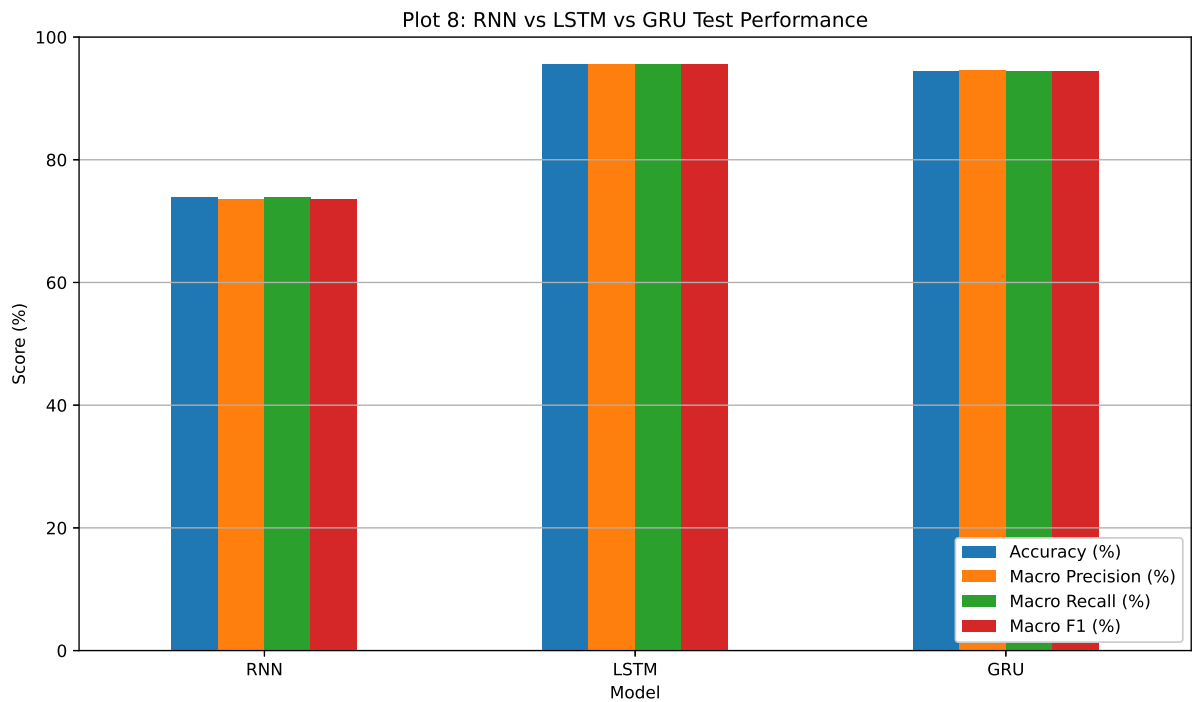


Figure 9: Model Performance Comparison

The plot compares the test performance of RNN, LSTM and GRU. LSTM and GRU show substantially higher accuracy and F1-score than the basic RNN. This improvement is associated with their gated mechanisms for handling temporal dependencies, while they require more parameters than RNN. Hence, the experiment demonstrates a trade-off between predictive performance, model complexity and computational cost.

Required inference:

Students should discuss the trade-off among:

$$\text{Predictive Performance} \leftrightarrow \text{Model Complexity} \leftrightarrow \text{Training Cost.}$$

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	75.088	91.076	91.356
64	70.157	86.227	93.904
128	69.687	69.951	95.016

Plot 6: Sequence Length vs. Test F1-score

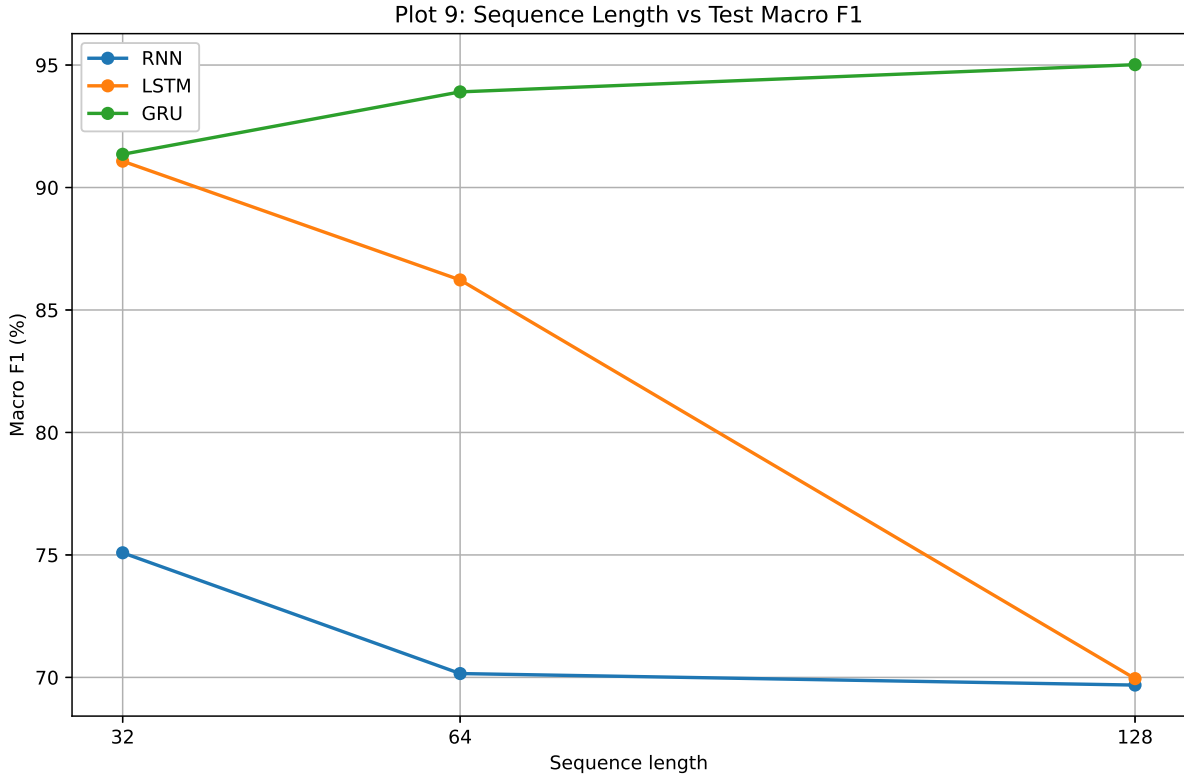


Figure 10: Sequence Length vs. Test F1-score

The plot shows how test F1-score changes with sequence lengths of 32, 64 and 128. Increasing sequence length provides more temporal context, which can help the model capture activity patterns more effectively, although it also increases computational requirements. Thus, sequence length influences both temporal information available to the model and classification performance.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

```
Predicted class : Basketball
Actual class    : Basketball
Confidence      : 0.xx
```

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

Required Observation:

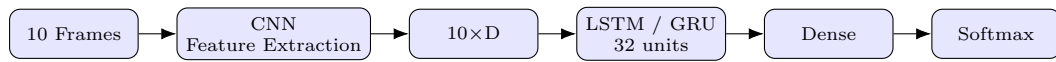
The CNN extracts a feature vector of dimension **1280** from each frame. These features are arranged as sequences of 10 frames and supplied to the recurrent network.

Table 2: CNN Feature Dimension and RNN Input Shape

Parameter	Value
CNN Feature Dimension	1280
Training Input Shape	(100, 10, 1280)
Validation Input Shape	(22, 10, 1280)
Testing Input Shape	(22, 10, 1280)

21. CNN-LSTM / CNN-GRU Model

Use the following architecture:



Plot 7: Video Sample Frames

Display a representative sequence of sampled frames from one video.

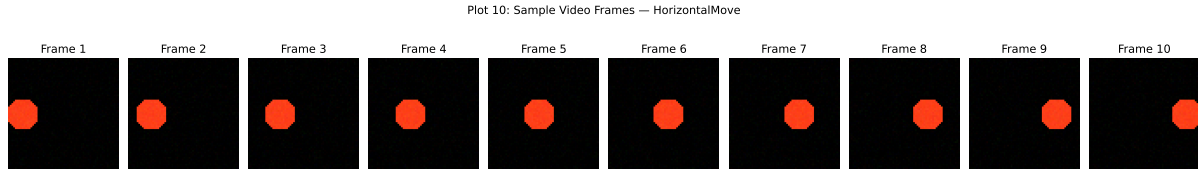


Figure 11: Video Sample Frames

The sampled frames show the progression of an action across time. The CNN extracts spatial information such as objects, shapes and visual patterns from individual frames, while the sequence of extracted features preserves the temporal order. Thus, the plot illustrates the input used for CNN-LSTM/GRU-based video understanding.

Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture.

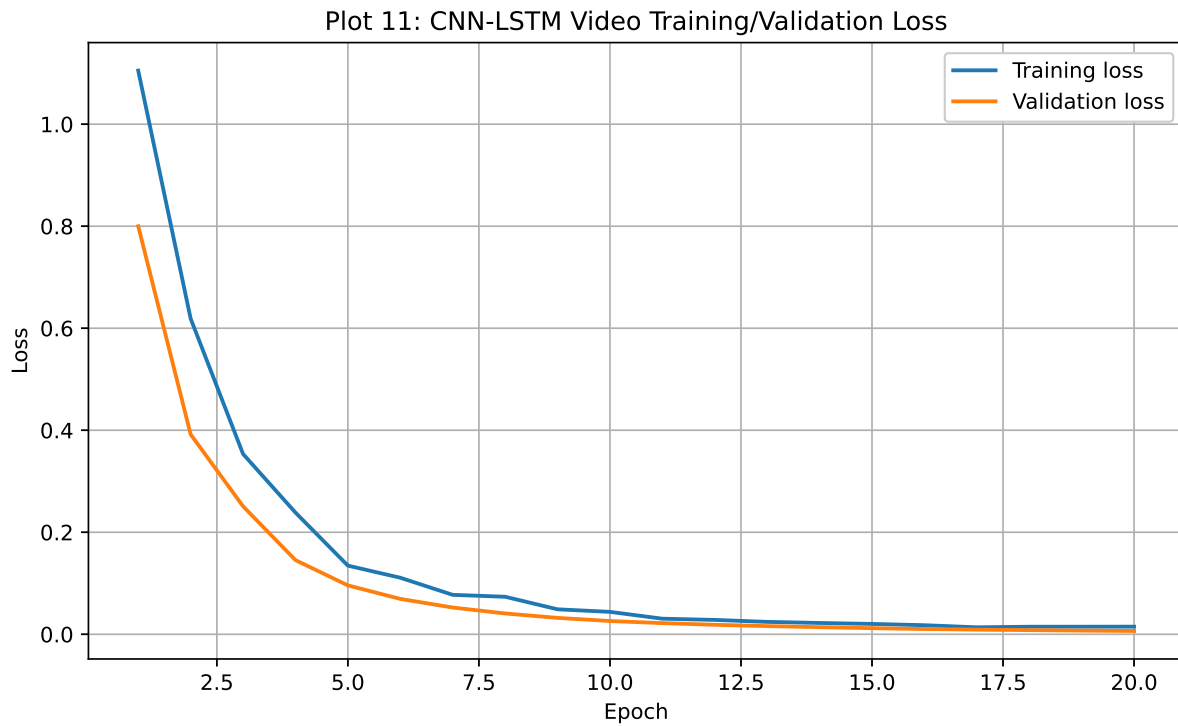


Figure 12: Video Training Loss

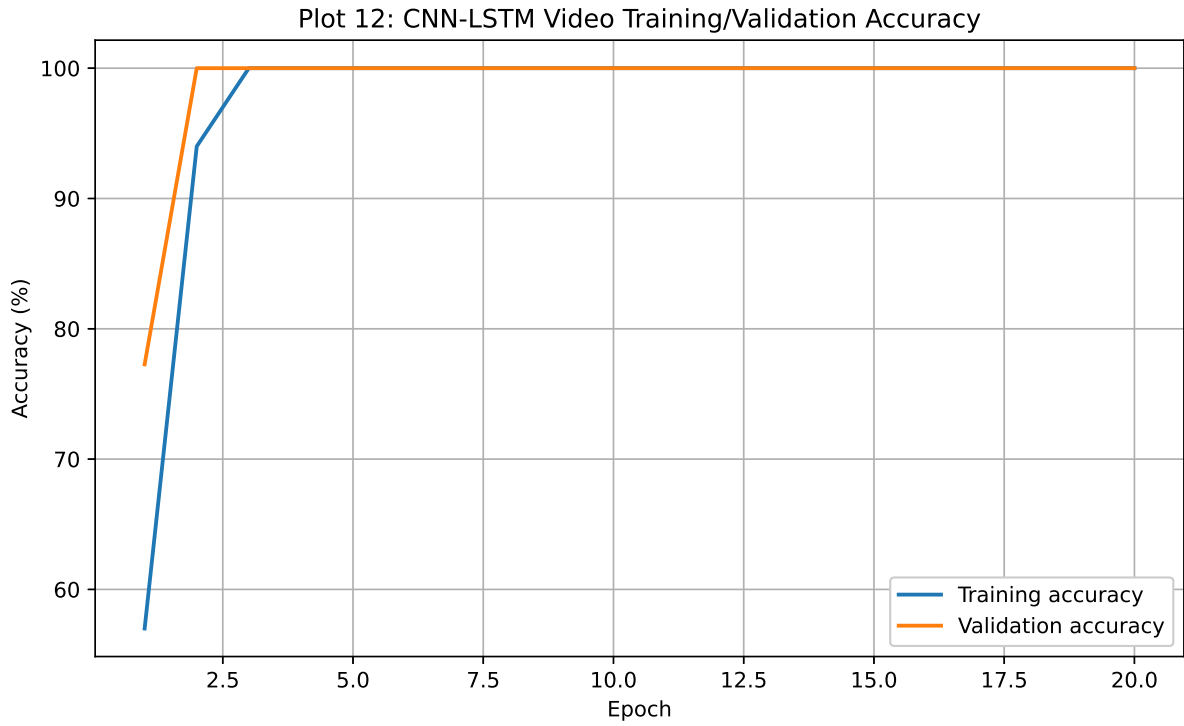


Figure 13: Video Training Accuracy

The training and validation curves show how the selected recurrent model learns from CNN-extracted video features over epochs. Decreasing loss and increasing accuracy indicate learning and convergence, while a widening gap between training and validation curves would indicate overfitting. The recurrent network therefore learns temporal relationships between features extracted from successive video frames.

Plot 9: Video Confusion Matrix

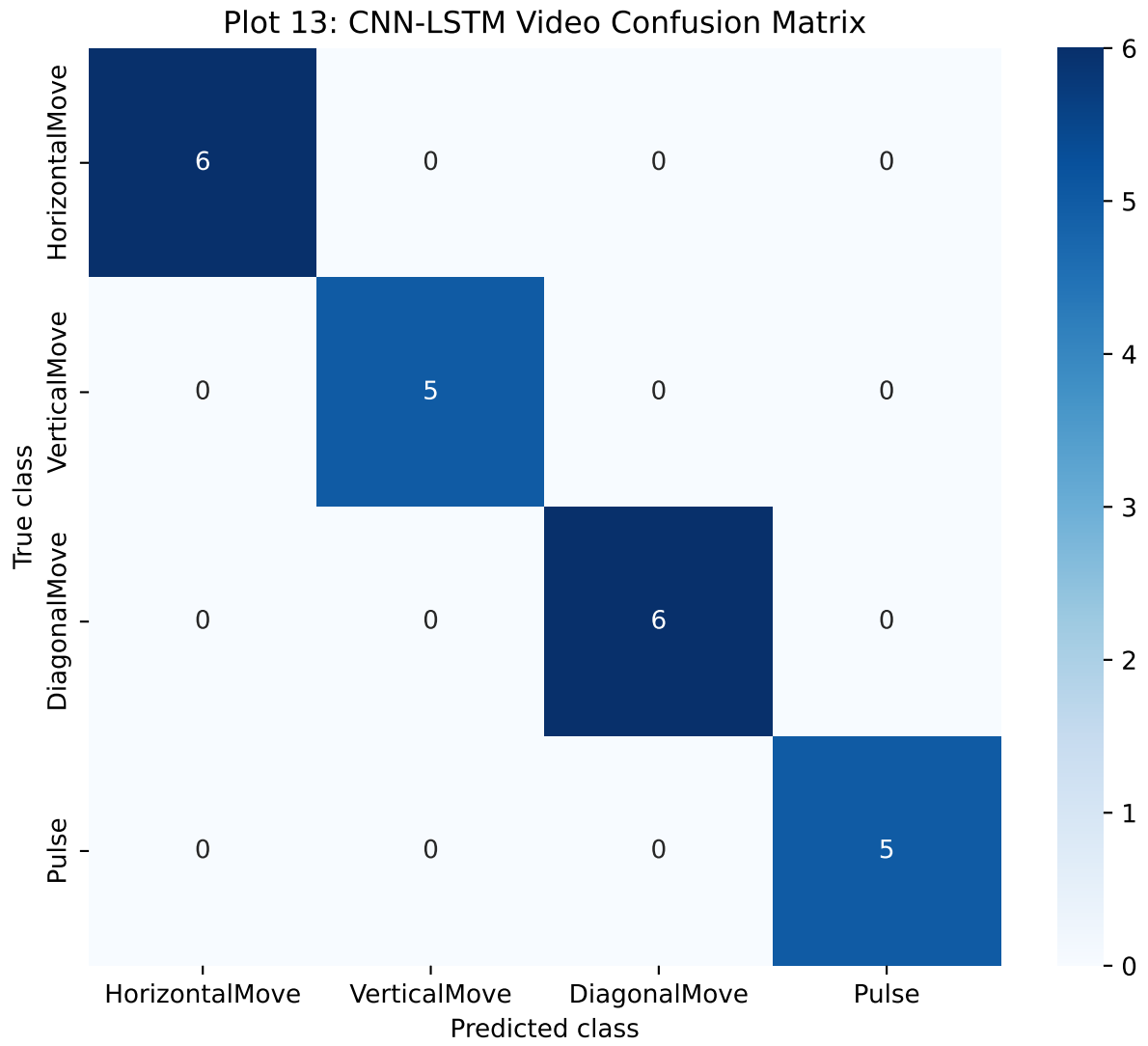


Figure 14: Video Confusion Matrix

The confusion matrix shows the class-wise recognition performance of the video model. Correct predictions appear along the diagonal, while off-diagonal entries represent temporal or visual confusions between action classes. These errors can occur when different actions contain similar frame-level features or motion patterns. Thus, the matrix helps identify which video activities are difficult to distinguish.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

[3, 8, 1, 5]

Expected output:

[5, 1, 8, 3].

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1	[4, 2, 2, 7]	[7, 2, 2, 4]
2	[7, 8, 9, 0]	[0, 9, 8, 7]
3	[0, 9, 5, 2]	[2, 5, 9, 0]
4	[1, 5, 8, 5]	[5, 8, 5, 1]
5	[5, 8, 1, 9]	[9, 1, 8, 5]

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

Answer:

Sequence-level accuracy can be lower than token accuracy because a sequence is considered correct only when **all tokens** are predicted correctly. Even a single incorrect token makes the entire sequence wrong. Therefore, small token-level errors can lead to a much lower sequence-level accuracy.

25. Consolidated Results

Students must complete the following table.

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	76.67%	75.04%	74.71%	74.63%	1974
LSTM	96.39%	96.67%	96.66%	96.66%	6006
GRU	96.39%	96.67%	96.66%	96.66%	4758
CNN-LSTM/GRU	—	—	—	—	—

Seq2Seq Results

Model	Token Accuracy	Sequence Accuracy	Training Loss	Validation Loss
Seq2Seq (reversal task)	100.0%	100.0%	0.001928	0.002393

A separate table must be provided for the sequence-to-sequence task.

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

Mandatory Inferences

1. Temporal Pattern in Sensor Signals

The sensor signals show distinct temporal variations for different activities. Walking shows more periodic fluctuations, while stationary activities show relatively stable patterns. This occurs because different physical movements produce different sensor patterns. Hence, the temporal order of sensor measurements is important for activity classification.

2. RNN Convergence

The RNN training and validation curves show gradual improvement in loss and accuracy, followed by stabilization, indicating convergence. The simpler recurrent structure can learn temporal patterns but may have limitations in capturing long-term dependencies.

3. LSTM Convergence

The LSTM curves show a decrease in loss and an increase in accuracy before reaching a stable region. Its gates and cell state help preserve relevant information across time steps, allowing it to learn long-term dependencies effectively.

4. GRU Convergence

The GRU training and validation curves show improvement followed by stabilization, indicating convergence. Its reset and update gates regulate temporal information with a simpler structure than LSTM, allowing effective sequence learning.

5. Overfitting / Underfitting

Overfitting is indicated when training performance improves while validation performance stops improving or worsens. Underfitting is indicated by low and closely matched training and validation performance. The training and validation curves can therefore be used to assess the model's generalization.

6. Activity-wise Errors from Confusion Matrix

The confusion matrix shows correct predictions along the diagonal and misclassifications in the off-diagonal cells. Activities with larger off-diagonal values are more frequently confused, usually because their sensor patterns are similar. Thus, the confusion matrix provides class-wise error information that accuracy alone cannot show.

7. Effect of Sequence Length

Changing the sequence length from 32 to 64 and 128 changes the amount of temporal information available to the model. Longer sequences provide more temporal context but also increase computational requirements. Therefore, sequence length affects both temporal learning and model performance.

8. Parameter and Computational Differences

RNN has the fewest parameters (1,974), while LSTM has 6,006 and GRU has 4,758 parameters. The additional gates increase the complexity of LSTM and GRU compared with RNN. The training times are 43.95 s for RNN, 79.16 s for LSTM and 115.32 s for GRU. Hence, recurrent models involve a trade-off between complexity, performance and computational cost.

9. Role of CNN in Video Understanding

The CNN extracts spatial features such as shapes, objects and textures from individual video frames. These features are then arranged as a sequence and passed to the recurrent model. Thus, the CNN performs spatial feature extraction.

10. Role of the Recurrent Model in Temporal Modeling

The recurrent model processes CNN features in their time order and learns relationships between consecutive frames. Therefore, the CNN captures spatial information, while the LSTM or GRU captures temporal information for video understanding.

11. **Token Accuracy vs. Sequence Accuracy**

Token accuracy measures the percentage of individual tokens predicted correctly, whereas sequence accuracy requires the complete output sequence to be correct. Therefore, a sequence may have high token accuracy but still be considered incorrect if even one token is mispredicted. Hence, sequence accuracy is a stricter measure of complete sequence generation.

27. Discussion Questions

1. **What is a recurrent neural network?**
An RNN is a neural network designed to process sequential data by using information from previous time steps.
2. **What information is represented by the hidden state?**
The hidden state stores relevant information from the current input and previous inputs.
3. **Why is sequence order important?**
The meaning of a sequence often depends on the order in which its elements occur.
4. **Explain the difference between a feed-forward network and an RNN.**
A feed-forward network processes inputs independently, while an RNN maintains information from previous time steps.
5. **Explain Backpropagation Through Time.**
BPTT trains an RNN by unfolding it over time and propagating the error backward through all time steps.
6. **Why can gradients vanish during BPTT?**
Repeated multiplication by small gradient values can make the gradient become extremely small.
7. **Why can gradients explode during BPTT?**
Repeated multiplication by large values can make the gradients grow excessively.
8. **What are long-term dependencies?**
They are relationships where an earlier input affects the output much later in the sequence.
9. **What is the role of the LSTM cell state?**
The cell state carries important information across time and helps preserve long-term dependencies.
10. **What are the forget, input and output gates in LSTM?**
The forget gate removes unnecessary information, the input gate adds new information, and the output gate controls the information sent to the next step.
11. **What are the reset and update gates in GRU?**
The reset gate controls how much past information is used, while the update gate controls how much new information replaces the old state.
12. **Compare LSTM and GRU.**
LSTM has three gates and a separate cell state, while GRU has two gates and uses a single hidden state.
13. **Why does GRU generally use a simpler gating mechanism than LSTM?**
GRU combines some LSTM functions into fewer gates, making its structure simpler and often faster to train.
14. **Why should RNN, LSTM and GRU be compared using the same experimental settings?**
Using the same settings makes the comparison fair and ensures that performance differences mainly come from the model architecture.
15. **What does the confusion matrix reveal that accuracy alone does not?**
It shows which classes are being confused and gives a class-wise view of model performance.
16. **Why should macro F1-score be reported for multi-class classification?**
Macro F1 gives equal importance to every class and is useful when class performance needs to be compared fairly.
17. **What is the significance of sequence length?**
Sequence length determines how many time steps the model considers when learning temporal patterns.
18. **Why can a CNN be used as a feature extractor for video?**
A CNN can extract useful spatial features such as shapes, objects and textures from video frames.
19. **What type of information is extracted by the CNN?**
The CNN mainly extracts spatial information from individual frames.
20. **What type of information is learned by the recurrent network?**
The recurrent network learns temporal patterns and relationships between consecutive frames.
21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?**
They are arranged in time order so that the recurrent network can learn how features change across frames.
22. **Explain the encoder and decoder in sequence-to-sequence learning.**
The encoder converts the input sequence into a representation, and the decoder uses it to generate the output sequence.
23. **What is teacher forcing?**
Teacher forcing means using the actual previous output as the input to the decoder during training.

24. **Differentiate token accuracy and sequence accuracy.**

Token accuracy checks individual predictions, while sequence accuracy requires the entire sequence to be predicted correctly.

25. **Why must the test set remain untouched during model selection?**

It prevents information from the test set influencing the model and gives a more reliable estimate of its final performance.

28. Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.

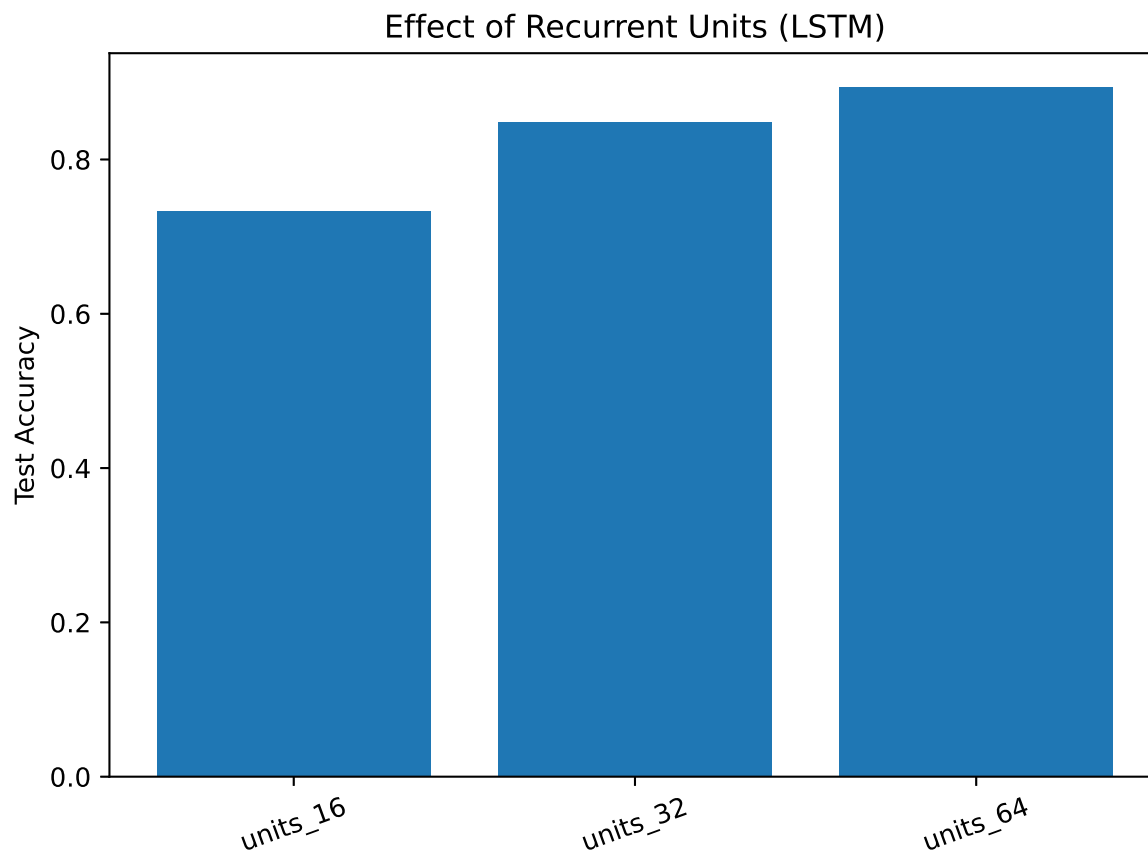


Figure 15: 16Units vs 32Units vs 64Units

2. Compare GRU and LSTM using the same number of recurrent units.

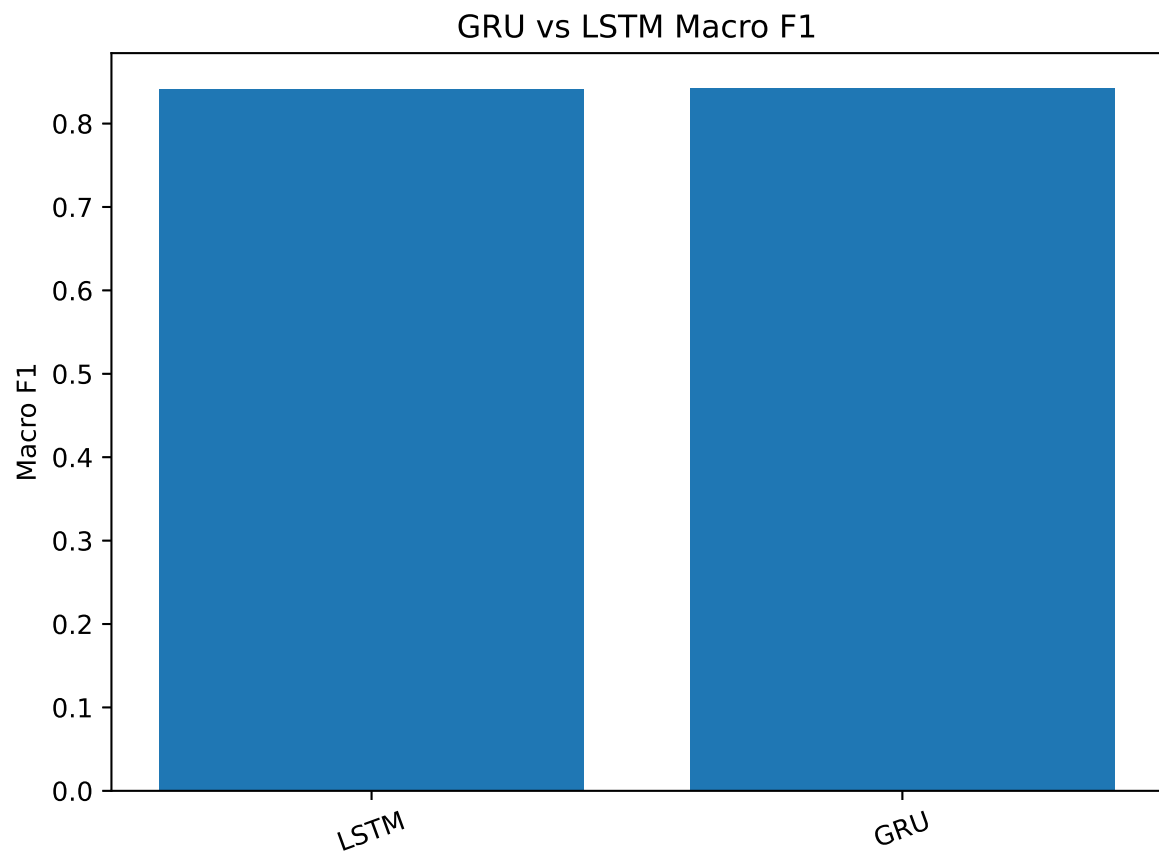


Figure 16: GRU vs LSTM Macro F1

3. Add a second recurrent layer and investigate whether performance improves.

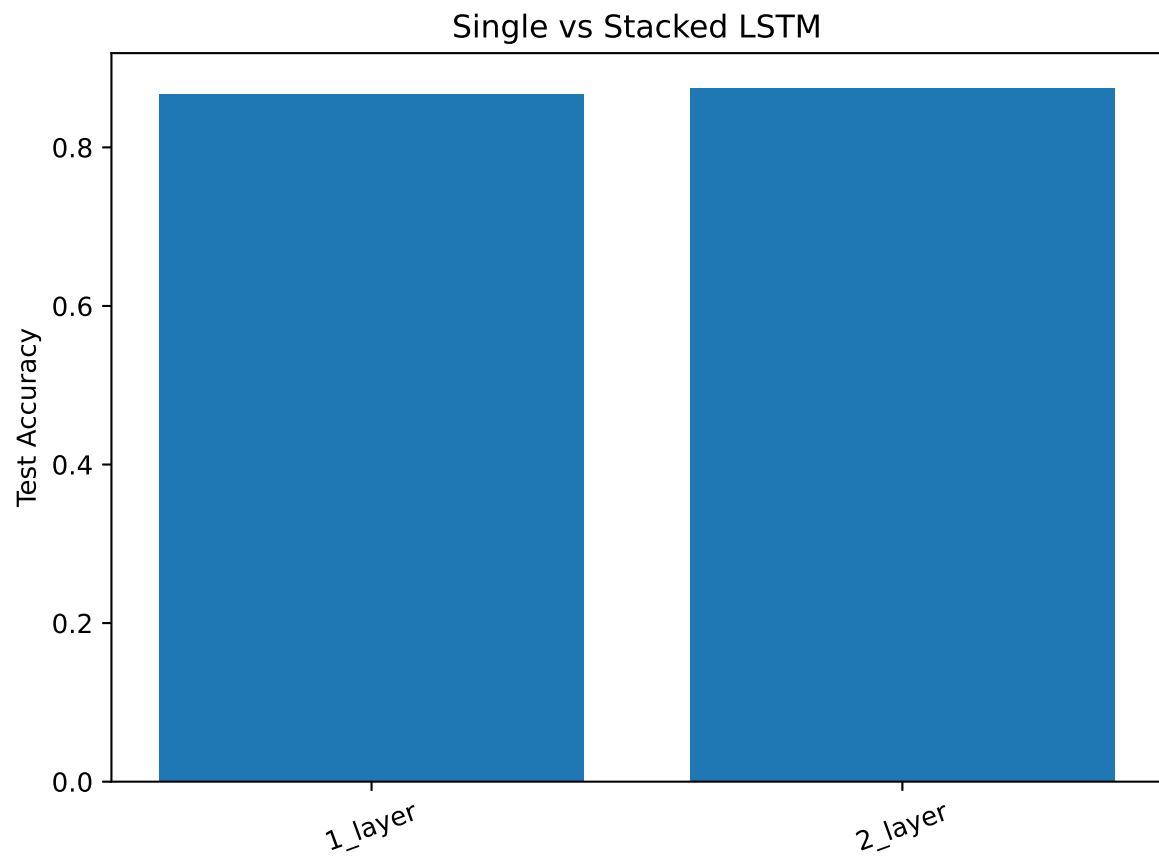


Figure 17: Single vs Stacked LSTM

4. Compare a bidirectional LSTM with a unidirectional LSTM.

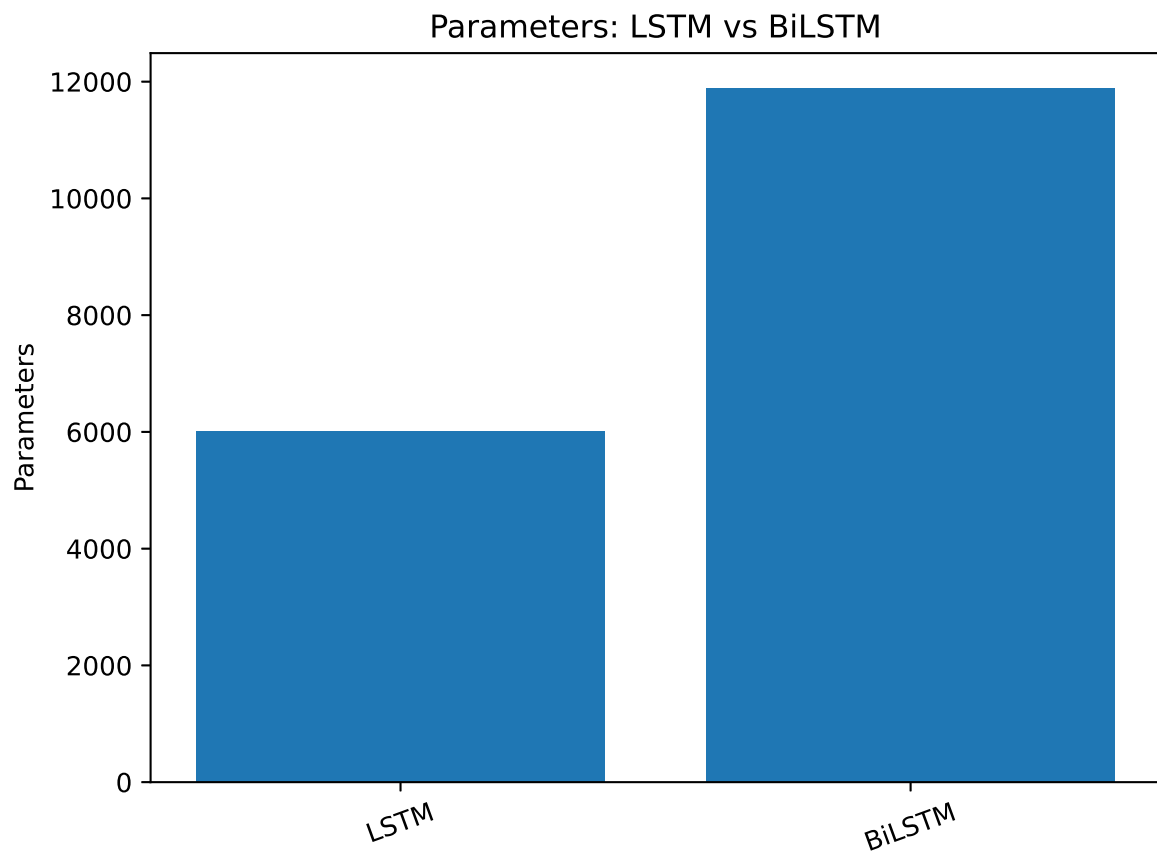


Figure 18: Parameters LSTM vs BiLSTM

5. Change the sequence length and study the effect on computational cost.

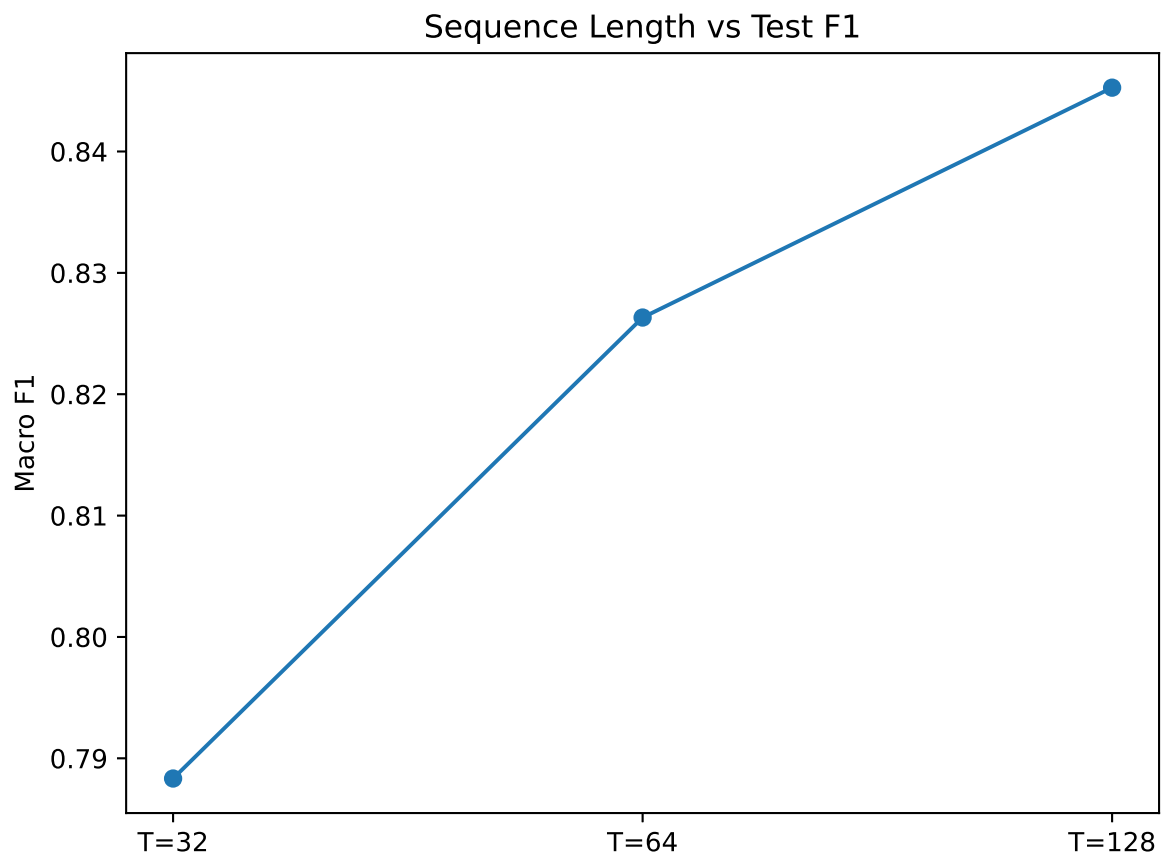


Figure 19: SequenceLength vs F1

6. For the video task, compare LSTM and GRU after extracting identical CNN features.

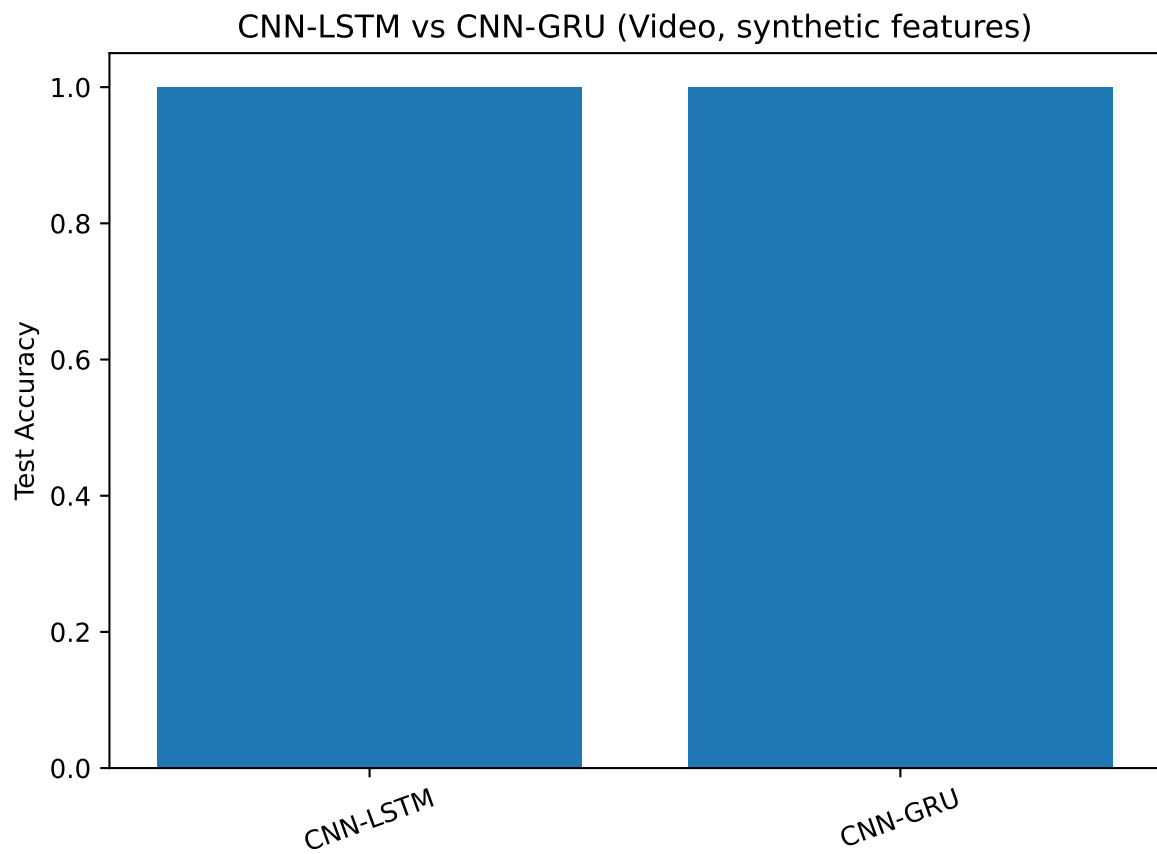


Figure 20: CNN-LSTM vs CNN-GRU

7. Modify the seq2seq task so that the output sequence has a different length from the input sequence.

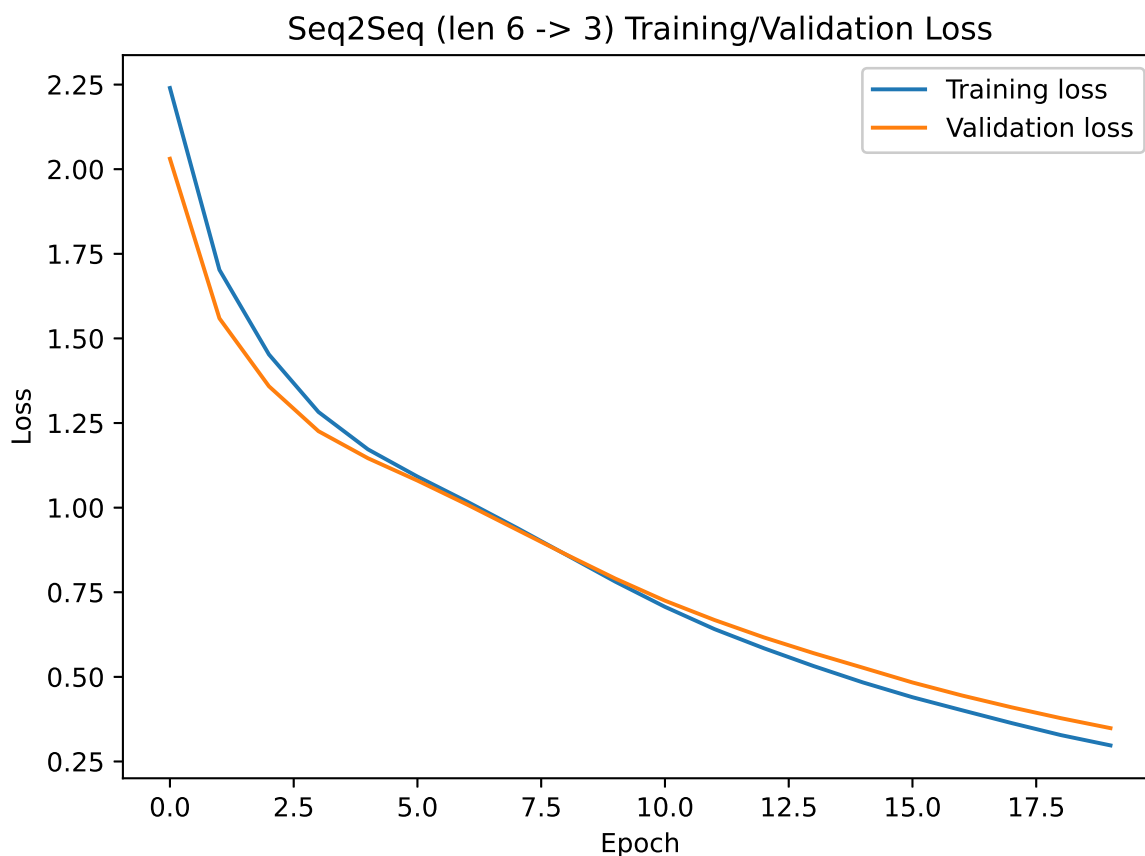


Figure 21: Seq2Seq(len6-3)

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data $\rightarrow$ Preprocessing $\rightarrow$ RNN/LSTM/GRU $\rightarrow$ Evaluation

and the video-understanding pipeline:

Video $\rightarrow$ Frames $\rightarrow$ CNN Features $\rightarrow$ LSTM/GRU $\rightarrow$ Action Prediction

They should also understand the encoder-decoder formulation:

Input Sequence $\rightarrow$ Encoder $\rightarrow$ Context $\rightarrow$ Decoder $\rightarrow$ Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student's own execution and should not be assumed in advance.

30. Source Code

The complete source code, datasets, generated plots, and report files for this experiment are available in the following GitHub repository:

GitHub Repository:

<https://github.com/VIKASNI-S/Deep-Learning-Lab/tree/main/Lab-6>

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.

2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>